```python
# KNN

import numpy as np
from collections import Counter

def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def knn(X_train, y_train, X_test, k):
    predictions = []

    for test_point in X_test:
        distances = []

        for i in range(len(X_train)):
            distance = euclidean_distance(test_point, X_train[i])
            distances.append((distance, y_train[i]))

        distances.sort(key=lambda x: x[0])
        k_neighbors = distances[:k]

        labels = [label for _, label in k_neighbors]
        most_common = Counter(labels).most_common(1)[0][0]

        predictions.append(most_common)

    return predictions

# Sample Data
X_train = np.array([[1,2], [2,3], [3,4], [6,7], [7,8], [8,9]])
y_train = np.array([0,0,0,1,1,1])

X_test = np.array([[5,5], [2,2]])

k = 3

result = knn(X_train, y_train, X_test, k)
print("Predictions:", result)
```

```
Predictions: [np.int64(0), np.int64(0)]
```

```python
# K Means

import numpy as np

def kmeans(X, k, max_iters=100):
    n_samples, n_features = X.shape

    # Initialize centroids randomly
    centroids = X[np.random.choice(n_samples, k, replace=False)]

    for _ in range(max_iters):
```

```
        clusters = [[] for _ in range(k)]

        # Assign points to nearest centroid
        for idx, point in enumerate(X):
            distances = [np.linalg.norm(point - centroid) for centroid
in centroids]
            closest = np.argmin(distances)
            clusters[closest].append(idx)

        # Store old centroids
        old_centroids = centroids.copy()

        # Update centroids
        for i, cluster in enumerate(clusters):
            if cluster:
                centroids[i] = np.mean(X[cluster], axis=0)

        # Check convergence
        if np.all(centroids == old_centroids):
            break

    return centroids, clusters

# Sample Data
X = np.array([[1,2], [2,3], [3,4], [6,7], [7,8], [8,9]])

k = 2

centroids, clusters = kmeans(X, k)

print("Centroids:\n", centroids)
print("Clusters:", clusters)
```

```
Centroids:
 [[2 3]
 [7 8]]
Clusters: [[0, 1, 2], [3, 4, 5]]
```